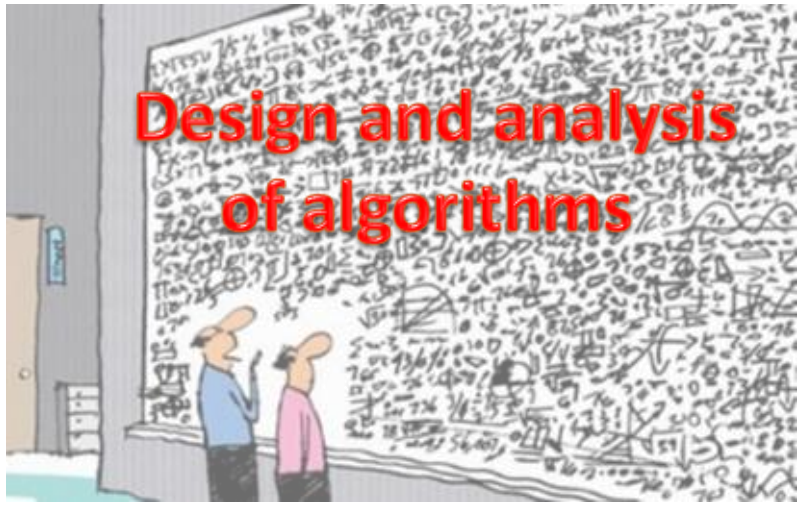


Greedy algs

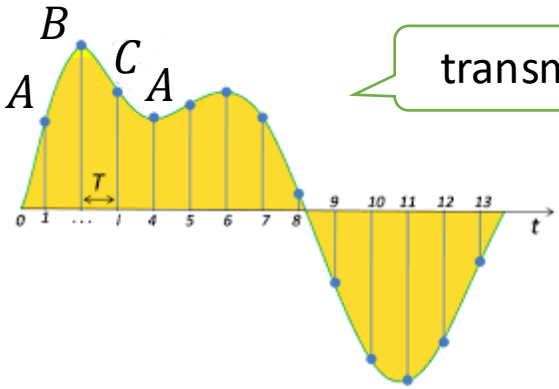
- Huffman encoding
- Min set cover

Objective

- Huffman encoding.
- Set covering.
- DPV Chapter 5 (we don’t cover the material in section 5.3).

Huffman encoding

- MP3: analog signal is digitized by sampling at regular intervals, yielding a sequence of real numbers $s_1, s_2, \ldots, s_T$.
- For instance, at a rate of 44,100 samples per second, a 50-minute symphony would correspond to $T = 50 \times 60 \times 44{,}100 \approx 130$ million measurements.
- Each real-valued sample s_t is quantized: approximated by a nearby number from a finite set Γ of "rounded" numbers.
- The resulting string of length T over alphabet Γ is encoded in binary.
- What is the most economical way to write this long string in binary?



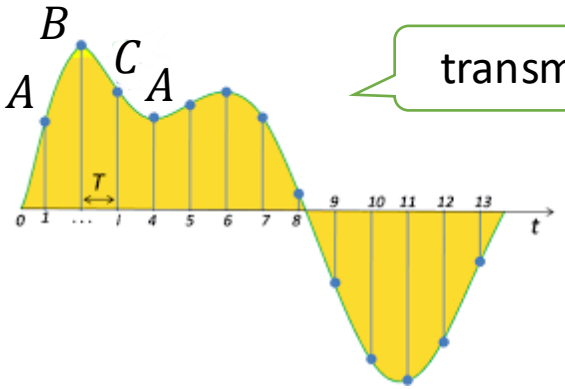
transmit: $ABCA$ …

for simplicity assume
4 possible values
$\Gamma = \{A, B, C, D\}$

Symbol	Frequency
A	70 million
B	3 million
C	20 million
D	37 million

A: 00, B:01, C:10, D:11
need $130 \cdot 10^6 \cdot 2$ bits

Huffman encoding



transmit: $ABCA$ …

- MP3: analog signal is digitized by sampling at regular intervals, yielding a sequence of real numbers $s_1, s_2, \ldots, s_T$.
- For instance, at a rate of 44,100 samples per second, a 50-minute symphony would correspond to $T = 50 \times 60 \times 44{,}100 \approx 130$ million measurements.
- Each real-valued sample s_t is quantized: approximated by a nearby number from a finite set Γ.
- The resulting string of length T over alphabet Γ is encoded in binary.
- What is the most economical way to write this long string in binary?

Symbol	Frequency
A	70 million
B	3 million
C	20 million
D	37 million

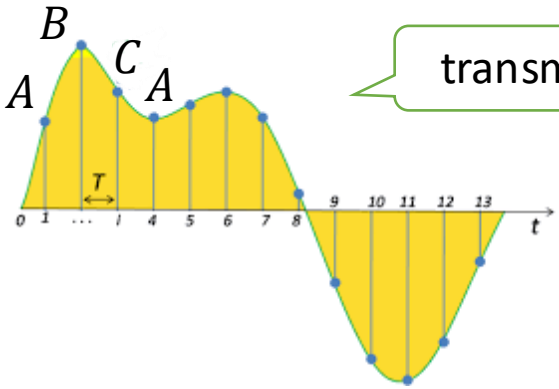
can we do better? use variable length encoding, i.e., shorter codewords for more frequent symbols…

A: 00, B:01, C:10, D:11

need $130 \cdot 10^6 \cdot 2$ bits

Huffman encoding

- MP3: analog signal is digitized by sampling at regular intervals, yielding a sequence of real numbers $s_1, s_2, \ldots, s_T$.
- For instance, at a rate of 44,100 samples per second, a 50-minute symphony would correspond to $T = 50 \times 60 \times 44{,}100 \approx 130$ million measurements.
- Each real-valued sample s_t is quantized: approximated by a nearby number from a finite set Γ.
- The resulting string of length T over alphabet Γ is encoded in binary.
- What is the most economical way to write this long string in binary?



transmit: $ABCA$ ...

Symbol	Frequency
A	70 million
B	3 million
C	20 million
D	37 million

can we do better? use variable length encoding, i.e., shorter codewords for more frequent symbols…

A: 00, B:01, C:10, D:11

need $130 \cdot 10^6 \cdot 2$ bits

need *prefix-free* property: no codeword can be a prefix of another codeword. Cannot have A = 01, B = 011

read the input without looking ahead

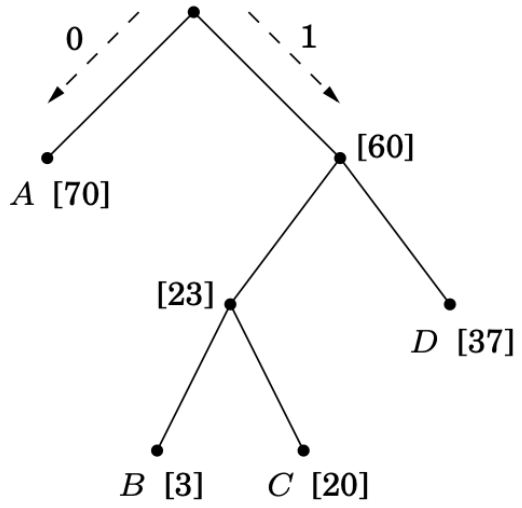
Any prefix-free encoding can be represented by a *full* binary tree.

Full binary trees

- A binary tree in which every node has either **zero** or **two** children.
- The **symbols** are **at the leaves**, and each codeword is defined by a path from root to leaf, interpreting left as 0 and right as 1.
- **Decoding is unique**: a string of bits is decrypted by starting at the root, reading the string from left to right to move downward the tree (left as 0 and right as 1), and whenever a leaf is reached, outputting the corresponding symbol and returning to the root.

Symbol	Codeword
A	0
B	100
C	101
D	11

need 213 Mbs, a 17% improvement



$f_1, f_2, \ldots, f_n$ = frequencies of n symbols

in millions

Total cost:

$= \sum_{i=1,..,n} f_i \times depth(i)$

$= \sum_{all\ nodes\ j\ of\ tree}$ frequency of node j .

Greedy algorithm

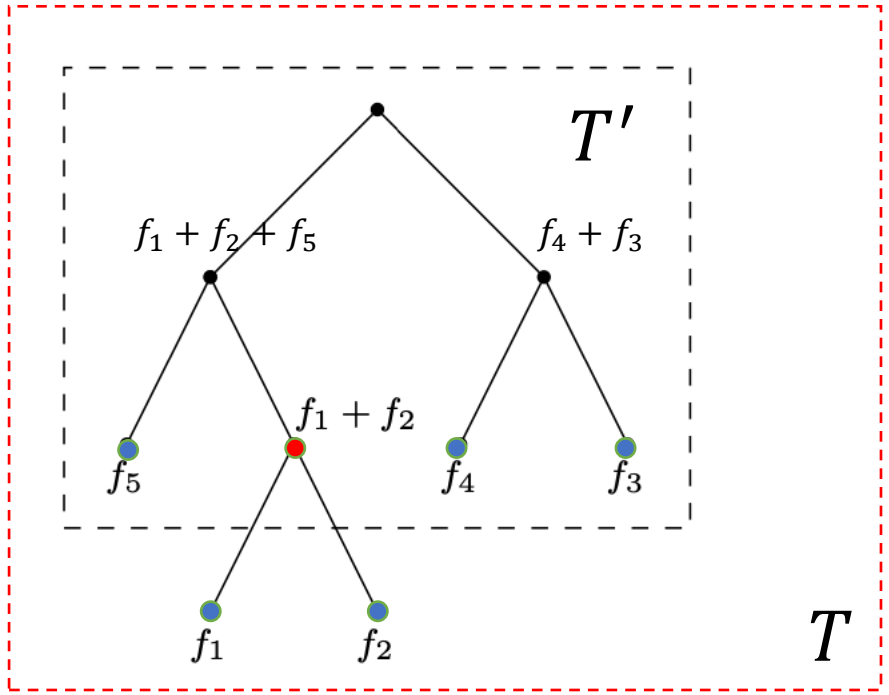
- Order $f_1 \leq f_2 \leq \cdots \leq f_n$.
- Optimal substructure: assume T optimal. Then:

a) Lowest 2-subtree has f_1, f_2.

b) Reduced tree $T' = T - \{f_1, f_2\}$ optimal for the $n-1$ numbers $\{f_1 + f_2, f_3, \ldots, f_n\}$.

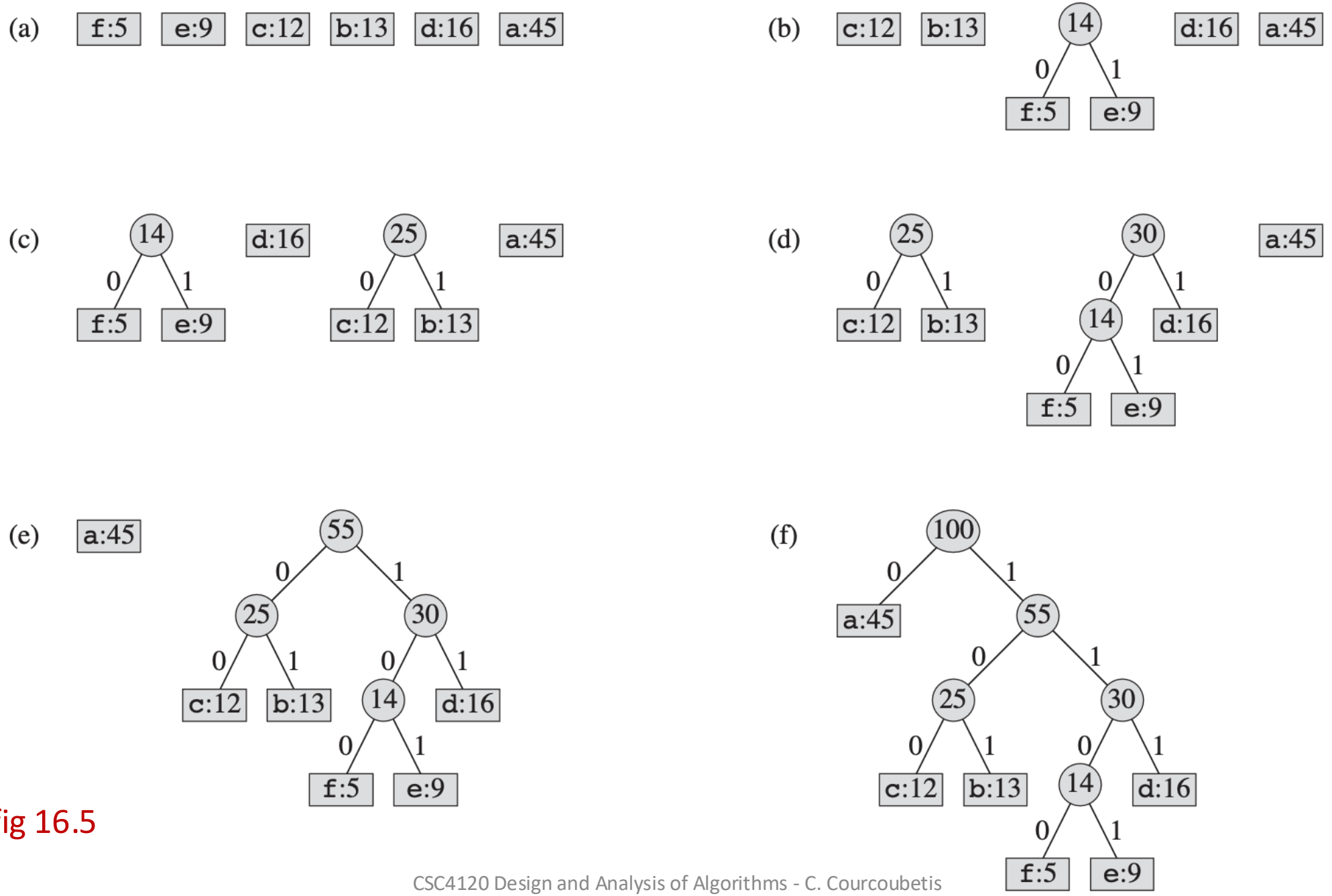
for $\{f_1, f_2, f_3, \ldots, f_n\}$

symbol 1+2



$$cost(T_{opt}) = f_1 + f_2 + cost(T'_{opt})$$

- **Algorithm**: build T bottom up: create subtree for two smallest f_1, f_2, then, for the two smallest numbers in $\{f' = f_1 + f_2, f_3, \ldots, f_n\}$, etc.


(a)
f:5
e:9
c:12
b:13
d:16
a:45
(b)
c:12
b:13
14
0
1
f:5
e:9
d:16
a:45
(c)
14
0
1
f:5
e:9
d:16
25
0
1
c:12
b:13
a:45
(d)
25
0
1
c:12
b:13
30
0
1
14
d:16
0
1
f:5
e:9
a:45
(e)
a:45
55
0
1
25
30
0
1
0
1
c:12
b:13
14
d:16
0
1
f:5
e:9
(f)
100
0
1
a:45
55
0
1
25
30
0
1
0
1
c:12
b:13
14
d:16
0
1
f:5
e:9

CLRS fig 16.5

Entropy

- The inherent unpredictability, or randomness, of a probability distribution can be measured by the extent to which it is possible to "compress data" drawn from that distribution (i.e., use fewer bits to transmit sequences of symbols).
- More compressible ≡ less random ≡ more predictable.
- Suppose there are n possible symbol outcomes $s_1, \ldots, s_n$, with probabilities $p_1, \ldots, p_n$.
- If we do (a large number) of m experiments, the resulting frequency of each symbol i is $f_i = mp_i$. How to encode each symbol to reduce the total # of bits to transmit the data?
- Shannon's theorem: we need at least $\log \frac{1}{p_i}$ bits for an outcome with prob. p_i, and to encode these m outcomes we need a minimum of

$$\sum_i f_i \log \frac{1}{p_i} = m \sum_i p_i \log \frac{1}{p_i} \text{ bits.}$$

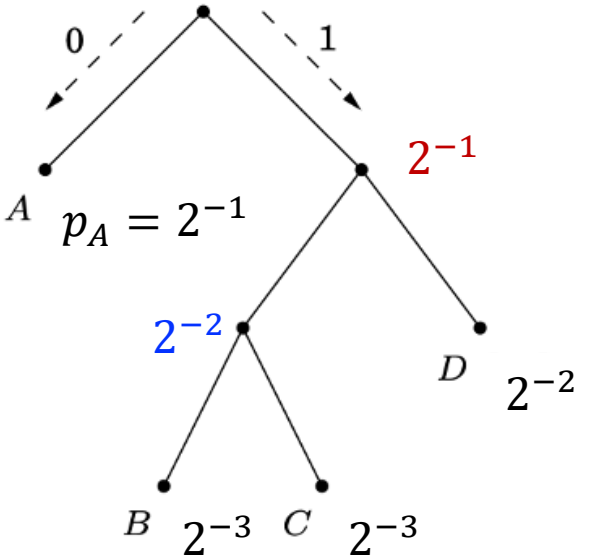
- **Entropy** of distribution: **average** number of bits per outcome $= \sum_i p_i \log \frac{1}{p_i}$.

$p_1 + \cdots + p_n = 1$

according to probability

Example

- Can we achieve # of bits for symbol $s_i = \log\frac{1}{p_i}$?
- Easy special case: assume that probs are of the form 2^{-k} and sum to 1. Then Huffman coding achieves the lower bound. See problem 5.19 in DPV (also in your homework).
- Example: $n = 4, p_A = 2^{-1},\ p_D = 2^{-2}, p_B = p_c = 2^{-3}$.
- $\{2^{-3}, 2^{-3}, 2^{-2}, 2^{-1}\}, \{2^{-2}, 2^{-2}, 2^{-1}\}, \{2^{-1}, 2^{-1}\}, \{1\}$.



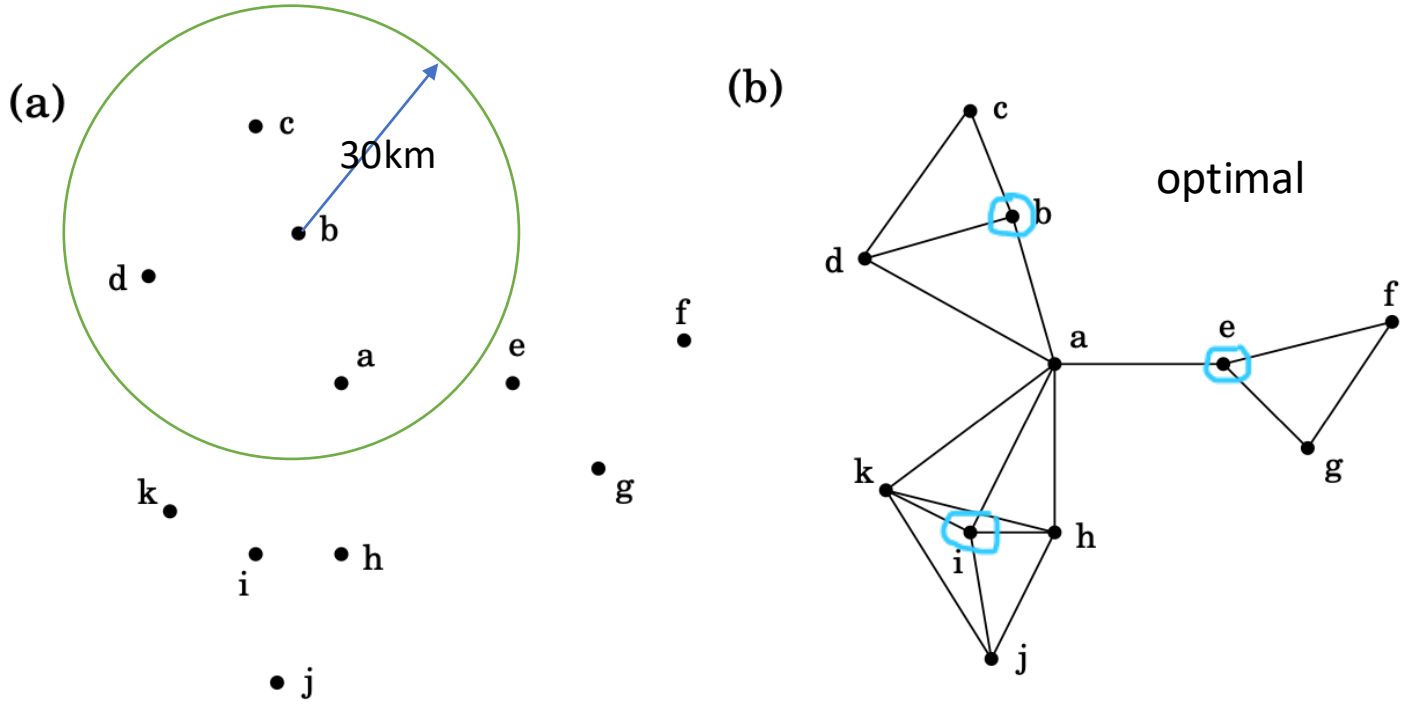
$\times m$ to get frequencies

The core idea of information theory is that the "informational value" of a communicated message depends on the degree to which the content of the message is surprising. If a highly likely event occurs, the message carries very little information. On the other hand, if a highly unlikely event occurs, the message is much more informative.

Huffman encoding achieves an encoding of $\log\frac{1}{p_i}$ bits for symbol with p_i.

Set cover

- Example of assigning schools to to towns: each school can serve students of towns within a 30km range. What is the smallest number of schools required and where?



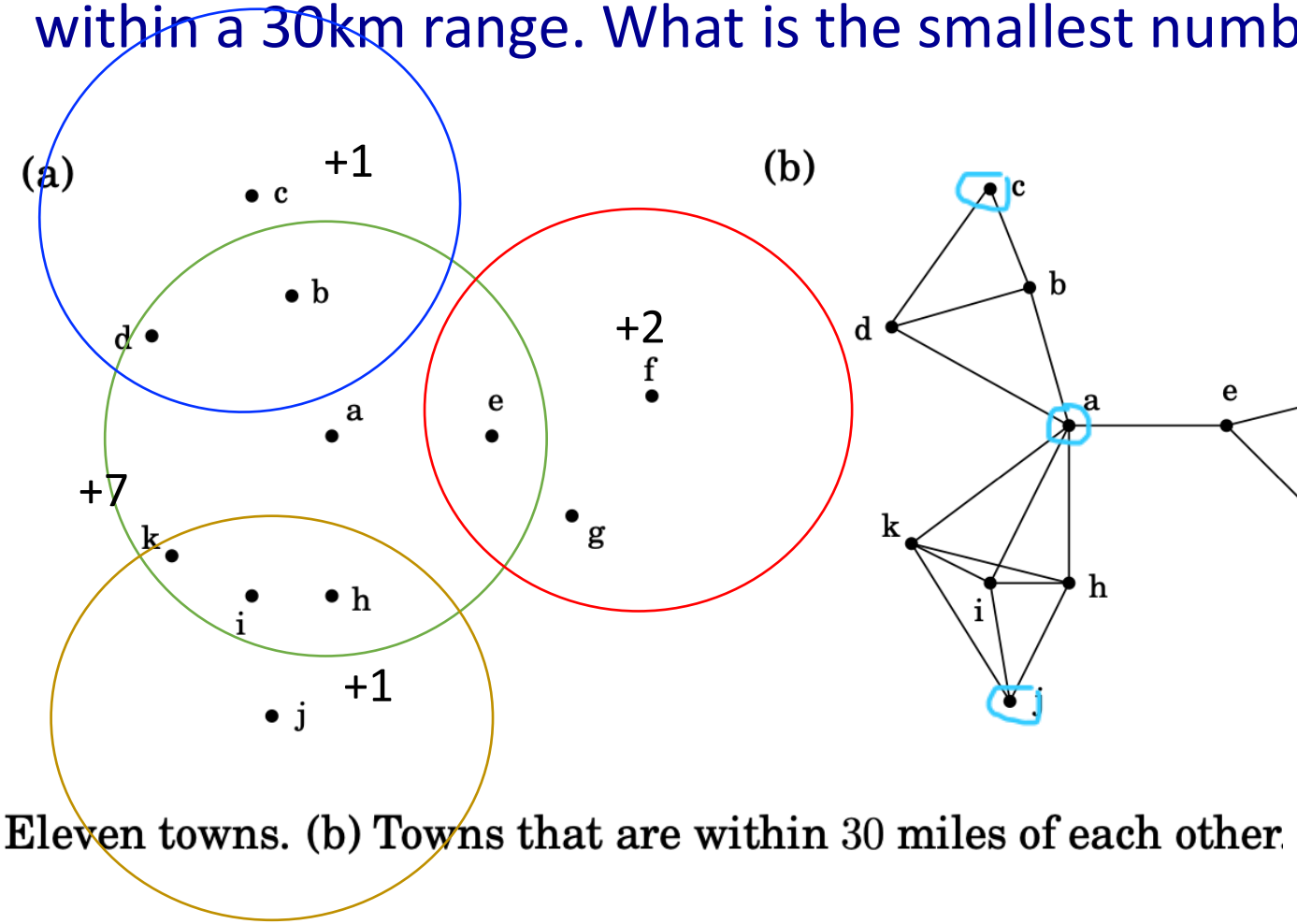
(a) Eleven towns. (b) Towns that are within 30 miles of each other.

Input: a set of elements B; sets $S_1, \ldots, S_m \subseteq B$.
Output: A selection of the S_is whose union is B.
Cost: Number of sets picked.

$B = \{a, b, c, d, e, f, g, i, j, k\}$
$S_a = \{a, d, b, e, h, i, k\}$,
$S_b = \{b, a, c, d\}$,
$S_c = \{c, d, b\}$,
$S_d = \{d, c, b, a\}$,
$S_e = \ldots$

Set cover

- Example of assigning schools to to towns: each school can serve students of towns within a 30km range. What is the smallest number of schools required and where?



(a) Eleven towns. (b) Towns that are within 30 miles of each other.

Greedy algorithm:

Repeat until all elements of B are covered:

- Pick the set S_i with the largest number of *uncovered* elements so far.

$S_a = \{a, d, b, k, i, h, e\}$

$S_g = \{f, g, e\}$

$S_c = \{c, d, b\}$

$S_j = \{j, i, h\}$

optimal: S_b, S_e, S_i

How well we approximate the optimum?

even if we don't know it!

- **Claim**: Suppose B contains n elements and that the optimal cover C^* consists of k sets. Then the greedy algorithm will use at most $k \ln n$ sets, i.e.,

$$\frac{C_g}{C^*} \leq \ln n.$$

- **Proof**: Let n_t be the number of elements still not covered after t iterations of the greedy algorithm (so $n_0 = n$).

Greedy alg. will ensure that:

$n_{t+1} \leq n_t - \frac{n_t}{k} = n_t(1 - \frac{1}{k})$.

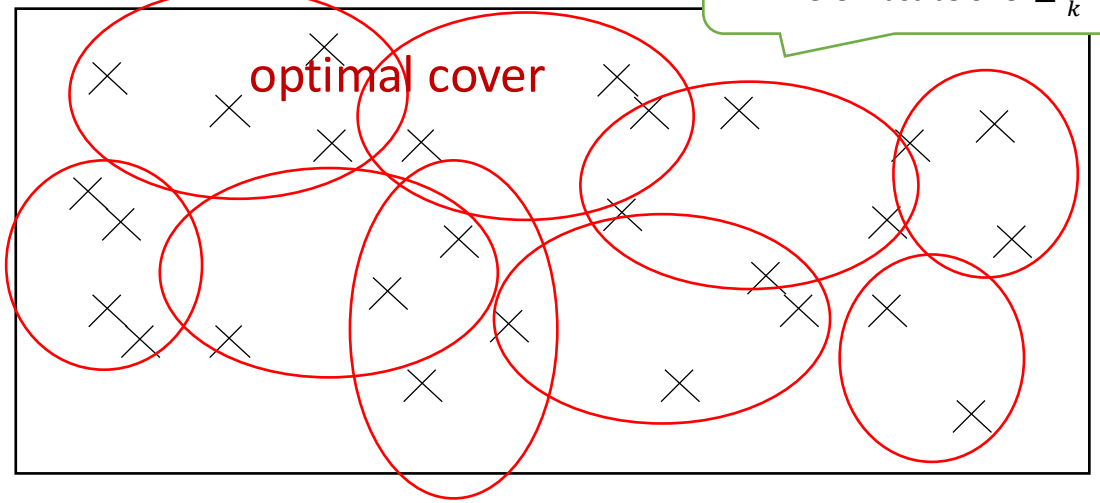
$x \neq 0$: $1 - x < e^{-x}$

Hence $n_t \leq n_0 \left(1 - \frac{1}{k}\right)^t < n_0 \left(e^{-\frac{1}{k}}\right)^t = n_0 e^{-\frac{t}{k}}$

At $t = k \ln n_0$, $n_t < 1 \Rightarrow$ everything is covered.

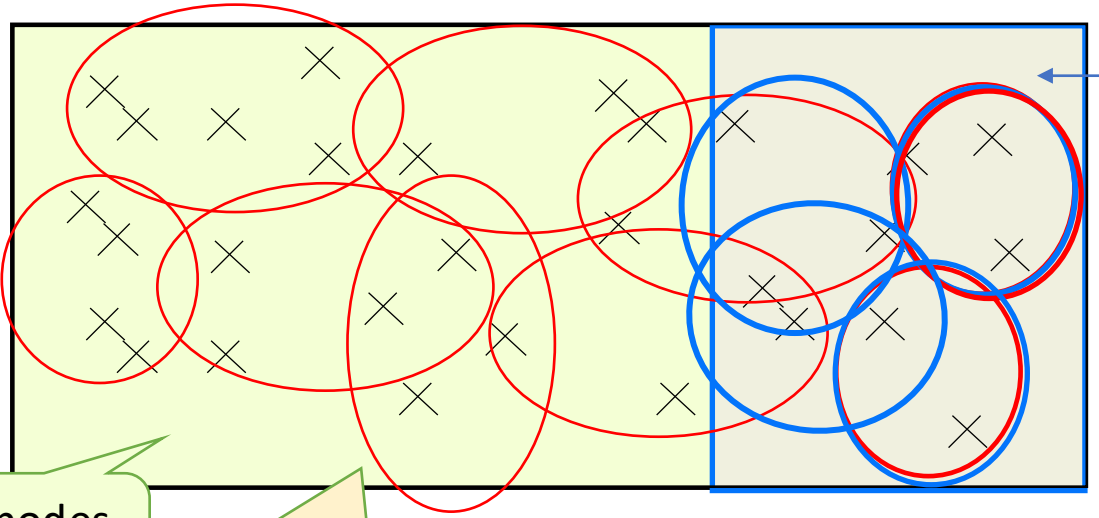
to understand why $n_{t+1} \leq n_t - \frac{n_t}{k}$ see next slide.

Same as: at step $t + 1$ we cover more than $\frac{n_t}{k}$ nodes.



Not all red sets can be small;
There must be one $\geq \frac{n}{k}$.

Total of n nodes.
Optimal cover $k = 9$ subsets, shown in red.
There must be a red set that covers more than the average cover, i.e., $\geq \frac{n}{k}$ nodes (otherwise, if each red set covers $< \frac{n}{k}$, there is no way that k sets cover all the n modes.)



n_t nodes remain

These are covered by less than k red sets.

Greedy cover (blue subsets) up to time t, n_t nodes remain to be covered. It has used (without knowing) 2 red subsets from the optimal set.
Fact: the remaining $k' = k - 2$ red subsets cover all the remaining n_t nodes and are available at t.
Hence, there must be a red set not used by the greedy up to time t (i.e., still available) that covers $\geq \frac{n_t}{k'} \geq \frac{n_t}{k}$ new nodes.
Hence, the greedy alg in step $t + 1$ will pick a set that covers at least what this red set can cover, i.e., cover $\geq \frac{n_t}{k}$ new nodes.

Conclusions

- Greedy algorithms are simple and intuitive.
- In many cases they are also optimal.
- If not optimal, we like to get an approximation ratio, i.e., a bound on how they compare to the optimum in the worse case.